

Persistencia en Árboles Rojinegros

Haskell, Clojure y Estructuras Funcionales

Integrantes:

[Nombre del Integrante 1]

[Nombre del Integrante 2]

[Nombre del Integrante 3]

Estructuras de Datos Avanzadas

11 de mayo de 2026

- **Árbol Rojinegro (RBT):** Árbol binario de búsqueda auto-balanceado.
- **Problema de la Mutabilidad:** En sistemas concurrentes, el cambio de estado "in-place" genera race conditions.
- **Solución:** Persistencia funcional mediante inmutabilidad y reparto estructural.

Propiedades Fundamentales del RBT

Para que un árbol sea Rojinegro, debe cumplir:

- 1 Todo nodo es o **Rojo** o **Negro**.
- 2 La raíz es siempre **Negra**.
- 3 Las hojas (nodos NIL) son **Negras**.
- 4 Si un nodo es Rojo, sus hijos deben ser Negros.
- 5 Cada ruta desde un nodo a sus hojas descendientes contiene el mismo número de nodos Negros.

El Concepto de Persistencia

Una estructura de datos es **persistente** si siempre preserva su versión anterior cuando es modificada.

Tipos de Persistencia

- **Parcial:** Solo se puede modificar la versión más reciente.
- **Total:** Todas las versiones pueden ser consultadas y modificadas.

¿Cómo logramos persistencia sin copiar todo el árbol?

- **Path Copying:** Solo se crean nuevos nodos para la ruta que va desde la raíz hasta el nodo modificado.
- Los nodos que no están en la ruta de modificación son compartidos por la versión antigua y la nueva.
- **Costo Espacial:** $O(\log n)$ por cada inserción o eliminación.

En Haskell, todos los datos son inmutables por definición.

- No existen los punteros modificables como en C.
- La recursión y el **Pattern Matching** sustituyen a los ciclos y condicionales complejos de balanceo.

Haskell: Definición del Tipo de Dato

```
data Color = R | B
data Tree a = E | T Color (Tree a) a (Tree a)

-- Insercion persistente
insert :: Ord a => a -> Tree a -> Tree a
insert x s = makeBlack (ins s)
where
  ins E = T R E x E
  ins (T col l y r)
    | x < y      = balance col (ins l) y r
    | x > y      = balance col l y (ins r)
    | otherwise = T col l y r
  makeBlack (T _ l y r) = T B l y r
```

Haskell: El Elegante Balanceo de Okasaki

En lugar de rotaciones manuales, usamos pattern matching para identificar los 4 casos de desequilibrio:

```
balance B (T R (T R a x b) y c) z d = T R (T B a x b) y (T B
c z d)
balance B (T R a x (T R b y c)) z d = T R (T B a x b) y (T B
c z d)
balance B a x (T R (T R b y c) z d) = T R (T B a x b) y (T B
c z d)
balance B a x (T R b y (T R c z d)) = T R (T B a x b) y (T B
c z d)
balance col l x r = T col l x r
```


Clojure: Persistencia Pragmática

Clojure utiliza **Persistent Data Structures** como base del lenguaje.

- Implementa `PersistentTreeMap` basado en Árboles Rojinegros.
- Utiliza una técnica llamada **Hashing** combinada con árboles para mayor eficiencia.
- Está diseñado para el manejo de estado en sistemas concurrentes masivos.

Clojure: Sorted Maps

```
;; Creacion de un mapa ordenado persistente
(def t1 (sorted-map :a 1 :b 2))

;; Generamos una nueva version (t2) sin alterar t1
(def t2 (assoc t1 :c 3))

(println t1) ; Output: {:a 1, :b 2}
(println t2) ; Output: {:a 1, :b 2, :c 3}
```

Nota: Ambas versiones comparten los nodos de :a y :b en memoria.

Ventajas de la Persistencia

Seguridad en Concurrencia

Múltiples hilos pueden leer el árbol sin necesidad de bloqueos (Locks), ya que los datos nunca cambian.

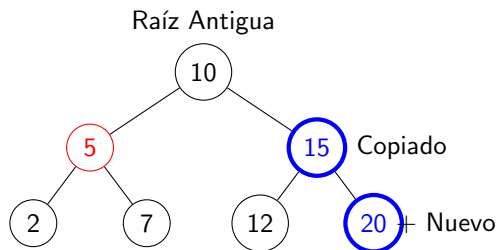
Persistencia Histórica

Permite implementar funciones de "Undo/Redo" de forma eficiente simplemente manteniendo punteros a raíces antiguas.

Operación	Tiempo	Espacio Adicional
Búsqueda	$O(\log n)$	$O(1)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$
Copia (Versión)	$O(1)$	$O(1)$

El costo espacial $O(\log n)$ es aceptable frente al beneficio de la inmutabilidad.

Visualización de Path Copying (TikZ)



Solo se recrean los nodos en azul; el subárbol izquierdo se comparte totalmente.

- Los RBT persistentes son la base de la programación funcional moderna.
- El **reparto estructural** hace que la inmutabilidad sea viable en términos de memoria.
- Lenguajes como Haskell y Clojure demuestran que el código es más seguro y fácil de razonar bajo este paradigma.

¿Preguntas?